Zanii Boundary: testing company-agent security boundaries

Authors: Louis Chifura, zaniiai
Date: 1 November 2026
Status: public engineering preprint; not peer reviewed

Abstract

Company agents need limits that remain enforceable when an agent follows malicious instructions. This paper presents a reference implementation for a declared document workflow: a trusted gateway authorizes exact actions, reserves durable operation state, keeps upstream credentials outside the agent, and dispatches to a resource that independently checks a sender-bound grant. Production admission additionally requires a recent signed record of explicit containment checks. An advisory detector can learn from independently reviewed examples without acquiring permission to change task authority.

The implementation now includes real TLS/mTLS, shared PostgreSQL authority, fenced maintenance, tenant/version-bound object encryption, independent audit/archive roles, resumable encrypted backup and quarantined recovery, bounded learning, protocol adapters and a strict Linux deployment profile. The actual upstream Runtime 0.4.0 executes a fixed reversible authorization preflight with accepted local ledger receipts; durable company effects remain governed by Boundary. Local tests use actual loopback TLS and SQL contracts with synthetic company data. Real Linux/container/provider/HA/customer acceptance remains incomplete. The contribution is a reproducible engineering design and measurement method; this paper establishes neither a new authorization primitive nor worldwide invention priority.

1. Problem and scope

Prompt injection can change an agent's proposed behavior. A prompt-only defense does not constrain a direct HTTP client, a child process or a stolen provider credential. The relevant question is whether every protected effect still crosses a correctly authorized boundary.

The first supported workflow reads protected company documents and shares them into a controlled internal mailbox. Tasks constrain tenant, actor, document set, recipients, purpose, action kinds, expiry, sensitivity, quota and delegation. There is no payment execution, clinical decision or physical actuator interface. Industry profiles are document-policy starting points and do not establish regulatory certification.

The runtime is model independent: company agents submit structured actions through a client or pinned protocol handler. The supplied JSONL runner is a narrow transport harness, not an autonomous planner or a model. Arbitrary existing company APIs require a trusted adapter and provider-side admission contract before they fall within the claim.

2. Prior work and research position

Primary source	Relevant established idea	Relation to this implementation
AgentSpec, ICSE 2026	Runtime triggers, predicates and enforcement for agents.	Runtime constraints and reviewed rule generation are prior work; a new policy gateway alone is not a novelty claim.
CaMeL: Defeating Prompt Injections by Design	Separate trusted control/data flow and constrain disclosure through capabilities.	Our document labels and destination checks share the objective of deterministic restrictions independent of model obedience. We do not reproduce CaMeL's planner or experiments.
Securing AI Agents with Information-Flow Control / Fides	Confidentiality/integrity tracking and deterministic policy checks.	Root-task sensitivity, untrusted memory and destination restrictions build on established information-flow thinking. They are narrower than general semantic tracking.
Formal Policy Enforcement for Agentic Systems, version 4	Event-graph-based runtime policy enforcement.	Current state and history matter at action admission; faithful observation and complete mediation remain important assumptions.
ACLE-MCP	Invocation-scoped, sender-constrained capability leases and resource-side admission.	Action binding, freshness and an independent resource gate already have close prior work. This implementation is not a reproduction or superiority evaluation of that system.
RFC 9334, RATS Architecture	Separate evidence, appraisal and relying-party decisions; reason explicitly about freshness.	A signed containment result is an authenticated observation, not a proof that the world cannot change immediately afterward.

These sources substantially overlap the broad architecture. The narrower research question is empirical: **which changes invalidate an agent-protection claim, and which controls prevent resulting effects while preserving authorized work?** A useful contribution can be an implementation, declared failure taxonomy and independently checkable measurements. Establishing novelty would require a systematic comparison, broader literature/patent work and independent review. None is claimed here.

3. Threat model and trust assumptions

An adversarial or tricked agent may submit malformed actions, access another tenant or task, change a recipient, replay an operation, request wider child authority, poison memory/feedback, or call a resource directly. Network delays, lost replies, restarts, stale evidence and changing document metadata are also modeled.

The trusted boundary includes the issuer and its key provisioning, gateway policy/configuration and storage keys, resource admission code/metadata, containment evaluator and its signing key, and the host/container security mechanisms when actually accepted. Host administrators, a compromised kernel/daemon, a compromised issuer or unrestricted provider credentials outside the declared route inventory are not repaired by the gateway.

A company must remove upstream credentials and privileged sessions from agent access, revoke legacy grants, restrict alternate provider routes, contain the workload and independently observe effects. If a provider continues to accept an old unrestricted credential directly, withholding a gateway grant cannot stop that bypass. The archived first milestone deliberately demonstrates this failure rather than reporting it as protected.

Finite negative probes cannot prove exhaustive route discovery. Signatures authenticate the evaluator's statement; they do not prove evaluator honesty, complete logs, hardware attestation or safety of permitted content.

4. Reference method

4.1 Explicit authority and durable reservation

The owner issues a task grant. Child tasks can only narrow scope, expiry, sensitivity and authority; every ancestor's revocation and aggregate quota are checked. Protected sharing requires a distinct approver and an exact one-use approval bound to task, operation, semantic arguments, policy digest, tool version and expiry.

Before any external effect, one scoped database transaction reserves a unique operation ID, consumes approval and charges the applicable ancestor quotas. Argument changes under an existing ID fail. The transaction never waits for network I/O. A crash or ambiguous response produces an unknown outcome; recovery observes that same operation rather than executing a replacement.

4.2 Credential custody and independent resource admission

The agent presents a short-lived runtime JWT and mTLS identity. The gateway pins issuer, audience, algorithm, lifetime and certificate thumbprint. A separate broker identity signs a resource grant binding tenant, task, action digest, operation ID, provider revision and deadline. Upstream credentials are injected only on a fixed trusted destination.

The resource independently checks the grant, tenant, document, destination, version, deadline and effect idempotency. Reads and committed internal deliveries are recorded in its own operation journal. The gateway reauthorizes cached or reconciled results before releasing content; A2A artifacts cannot bypass later revocation or changed document sensitivity.

When configured, the actual installed Zanii broker/SDK persist and obtain acceptance of a signed receipt before the protected POST. A post-recording barrier rechecks authority/deadline before I/O. Receipt recovery resubmits exact durable receipts without replaying company operations. Log inclusion is kept distinct from external/blockchain anchoring.

4.3 Coverage and containment

The Linux profile places the agent in a no-network container with a read-only narrow identity mount and authenticated gateway socket. Roles have separate keys/data mounts, non-root users, dropped capabilities, bounded resources and read-only roots. The supplied gateway/resource backend is internal; external SaaS/ledger/model connectivity needs a separately reviewed egress profile.

The checker declares ten configuration/positive/negative checks covering the actual image/container controls, authenticated path, network/backend reachability, metadata, DNS, secrets, runtime socket, socket directory and child inheritance. A fixed public diagnostic resolves bootstrap while company-data admission remains blocked. Its scope authorizes no selectable company document.

Production checks a signed, profile/image-bound receipt with at most 15 minutes' accepted age. Missing, altered or expired receipts fail closed in both gateway and resource. Provider revision preconditions and post-await authority checks constrain relevant dispatch races. There is no zero-delay claim for unobserved host/IAM changes: receipt freshness and complete live drift detection are different properties.

Static configuration validation is available locally. Real Linux UDS and container/network/secret-denial acceptance have not been established in the current Windows environment.

4.4 Non-escalating adaptive warnings

The baseline detector emits bounded instruction-override, authority-spoofing, disclosure and task-deviation warnings. A tenant-local exemplar component can improve similarity detection after review. This is not a new general prompt-injection algorithm.

Suspicious read content can be automatically quarantined with provenance, quotas and bounded redaction. A separate reviewer supplies labels and train/holdout splits. Training persists model, corpus and evaluation digests; staging runs in shadow. Promotion/rollback requires a separately signed, expiring, one-use update from an independent authorized actor. Failed evaluations, contaminated exact duplicates, wrong keys, stale policies and self-promotion are rejected.

The current promotion digest covers built-in authority profiles, not complete issuer/provider/deployment configuration. Hard execution checks remain independent. Learned examples and model updates cannot add issuers, scopes, recipients, budgets or approvals. Redaction can miss arbitrary secrets or personal information; examples remain local/encrypted and expire, and source deletion invalidates dependent candidates.

4.5 Evidence, retention and recovery

Private objects/results are AES-256-GCM encrypted with tenant/namespace/identifier-bound associated data. Audit ciphertext has a chained hash/HMAC structure. Signed bounded export pages can be independently verified using only a pinned public key: signatures, row hashes, predecessor links, tenant, sequence, page continuity and final checkpoint are checked. An independently retained expected checkpoint is required to detect substitution with an older valid snapshot.

A local SQLite database and its application signing key are not WORM storage. The extended implementation adds an independent keyless archive reader/signing identity, actual S3 SDK immutable-version/retention checks and protected audit-prefix commits after remote verification. Those TLS contract tests do not establish genuine cloud IAM/Object Lock or independent checkpoint/key custody. Protected audit exports still contain identifiers, timestamps and ciphertext, so they are not anonymous public artifacts.

Retention scrubs expired payloads while preserving no-replay barriers and unresolved-effect evidence. Typed privacy tombstones, holds, exports, object purge seals and reviewed-learning cleanup are integrated; company category policy still requires acceptance. PostgreSQL backups use bounded

consistent per-tenant logical snapshots, immutable pinned catalogues and encrypted resumable spools. New-target restore remains frozen/quarantined with old task/approval authority revoked. Actual source fencing, signed privacy reconciliation bound to the exact targetPrivacyDigest, retained effects and fresh grants must be reconciled before activation. This is not unlimited whole-cluster DR or automatic resumption of an old quota history.

5. Implementation and integration boundary

The implementation uses Node 24.19.0, native SQLite, JOSE, Zod and pinned Zanii packages. MCP is pinned to 2025-11-25 and A2A to 0.3.0. Both expose structured document actions through the same executor; free text does not become arbitrary authority.

Register clients follow public assurance/protected cookie contracts. Passport distinguishes verification with `dispatchPermit:false` from workload DPoP-authorized PEP execution. Vault uses its actual session/CSRF and declare/chunk/complete flow. Runtime-tenant binding protects shared configured companion credentials.

These clients are tested against real HTTPS contract fixtures using synthetic identities. Runtime 0.4.0 is now installed and called on the configured product path for a fixed reversible read-only preflight. It uses strict owner/manifest/delegation constraints, an exact action binding and actual accepted local TLS ledger receipts, with final Boundary authority/deadline checks preserved. Root SDK0.5.4 and Runtime nested SDK 0.6.0 are separately pinned. This scope does not register arbitrary company tools in the upstream runtime or replace durable Boundary authorization. Live customer/companion/ledger acceptance remains unestablished.

Node and Python clients verify TLS/mTLS, bound outputs and preserve uncertain outcomes without automatic mutation retries. SQLite is a bounded local backend; shared PostgreSQL authority, RLS, leases and maintenance roles are implemented and tested with genuine local SQL contracts. Independent PostgreSQL wire/HA sessions, actual provider policy and universal company-system compatibility remain outside the accepted deployment.

6. Historical pre-gap evaluation

The historical pre-gap product experiment used two actual TLS service roles and encrypted files in one Node process, with synthetic documents and PKI. It measures HTTP outcomes against independently queried provider effects. The fixed cases include useful read/share tasks, permission/tenant/recipient/scope/direct-resource negatives, response loss after a real commit, duplicate requests, restart, observation-only reconciliation, revocation and expiry.

The selected learning experiment has eight reviewed training examples and eight held-out examples. It exercises quarantine, review, training, shadow and signed promotion while checking unchanged task authority. Author-selected splits and exact-duplicate checks do not establish unseen-attack generalization or poisoning resistance under an independent adversary.

The historical pre-gap run `a0a30db5-3efb-4436-8bad-7a1a88f7106b` recorded **15/15 declared cases**, **2/2 authorized primary tasks**, **three expected provider effects**, **zero negative-case effects** and **zero duplicate effects**. The effects were the authorized read, approved internal-mailbox share and deliberately uncertain read; replay, restart and reconciliation produced no additional effect. Signature and

independent outcome/summary recount verification passed. Its historical source digest was `8478c9ac636aa9cedd40d561dca79c2d48de72a8f334b01741f4a5dc36516a62`, covering 66 implementation/configuration files. That historical checkpoint passed 130 Node tests with zero skips and one Python TCP/mTLS SDK test.

After eight reviewed training examples, the eight selected holdouts produced four true positives, four true negatives, zero false positives and zero false negatives. Signed promotion was exercised and the authority policy remained unchanged. These counts describe this author-authored split only; they are not an estimate of production accuracy or an adaptive-attacker evaluation.

A separate operations check measured a 45,056-byte synthetic encrypted snapshot: backup took approximately 2.51 ms and restoration approximately 5.54 ms. The restored payload/tombstone and externally pinned hash were checked, as were signed-chain verification, tamper rejection and retention/replay behavior. These small local timings do not establish off-site disaster recovery or a production RTO.

The load script issued **512 requests across four fresh fixtures**, recording **400 successful responses**, **112 admission throttles (429)** and **400 provider effects across those fixtures**. Operation IDs are unique within each fixture and reused across levels; uniqueness is not global across the four runs. There were no duplicated effects within a fixture, no pending/other-denied/error responses and no effects without a successful response.

Concurrent clients	Successful / requested	429	Successful requests/s	Successful p50 / p95 / p99, ms
1	128 / 128	0	27.33	34.13 / 52.30 / 65.72
4	128 / 128	0	27.60	137.72 / 174.38 / 190.38
16	128 / 128	0	23.55	675.89 / 719.37 / 728.87
64	16 / 128	112	12.34	1293.14 / 1296.98 / 1296.98

The gateway's configured 16-request admission limit was reached. Maximum observed successful throughput was **27.60 requests/s**; a hardware or production capacity ceiling was not established. Post-level combined RSS ranged from approximately 180 MB to 230 MB and includes evaluator plus both service roles. This short closed-loop synthetic workload is not a soak, a customer peak or per-service utilization measurement. Its illustrative 10 requests/s and latency targets are experimental assumptions, not an agreed SLO; p50 exceeded the illustrative 250 ms target at higher concurrency. No monetary billing, autoscaling or sustained production ceiling was measured.

6.1 Current implementation and separated measurements

The frozen implementation is `960c36c0c51d68b63351b9776549d7cfa8d191d0da2cb7320296572701d87ba4`, covering 190 implementation/configuration inputs. Broad verification passed 286/286 Node tests with zero skips in 72.169 seconds. A separate Runtime 14/14 run (8.151 seconds) includes two late additions and 12 overlapping cases; counts are not additive. Python 3/3, clean 116-file lint/checkJs and 153 parsed modules/five narrow rules passed. These supersede the historical implementation checkpoint without making its old load figures current.

An earlier full-feature warmup exposed normal local-writer acknowledgement contention. The corrected authorization barrier waits only behind an existing local writer tail under the deadline. Unknown peer/orphan pending evidence remains fail closed; distributed writer availability and multi-replica same-DID throughput remain unaccepted.

On the final source, the original **5 requests/s full-feature warmup was INVALID**: 25 issued, 20 successes, five RESOURCE_GRANT_EXPIRED, 20 effects and zero duplicates. Runtime authorized 25 preflights and the ledger accepted 70 receipts. Fixed three-second Runtime/short-grant budgets were unchanged; expiry guards held. Its intended 10 requests/s sustained cohort was never reached. This is a known local performance/usefulness gap.

The exploratory full-feature characterization declared a separate 0.5 requests/s planning basis. Its 1 request/s, 60-second sustained and 120-second soak cohorts completed 60/60 and 120/120; a 2 requests/s, 15-second observation completed 30/30. The 210 effects matched 210 authorized Runtime calls and 630 accepted local receipts, with zero drops, HTTP 429, errors or duplicate/unexpected effects. However, p50 was 343.6 / 310.9 / 330.4 ms, exceeding the unchanged 250 ms target in every cohort. Corresponding p95/p99 were 491.6/519.1, 474.3/685.6 and 441.0/455.4 ms. Configured 3/4 requests/s steps were not reached. Lower-rate observations do not close the original 5/10 requests/s objective.

Separately, core-only 10 requests/s sustained completed 600/600 in 60 seconds. Its 120-second soak issued 1,184 of 1,200 arrivals: 1,180 successes/effects, four HTTP 429 responses and 16 scheduling drops. At 20 requests/s, 290 of 300 arrivals were issued, with 194 successes/effects, 96 HTTP 429 responses and 10 drops. Oracles found no duplicate/unexpected effects; the first constraint was soak driver scheduling lag. All resource observations combine driver/gateway/resource in one process with local SQLite. No hardware or production capacity ceiling is established.

Fresh canonical evaluation run bc3bc612-713b-48c9-94f4-80cfa38ef071 passed 15/15 cases and 2/2 primary tasks, with three effects and zero negative/duplicate effects. Its selected synthetic holdout had 4 true positives, 4 true negatives and no errors. The canonical 512-request closed-loop sweep recorded 400 successes, 112 admission throttles and 400 effects; its maximum successful rate was 14.1766 requests/s, limited first by configured gateway concurrency 16. **Both canonical workloads use the default core-only fixture**, without Runtime/local-ledger receipts. Runtime evidence comes from separate actual-module tests and characterization. See the performance limits and evidence scope.

The cost model uses exact BigInt prices/quantities, dated quote metadata, explicit rounding and reconciled usage. No real provider quote, billed-host plan or billing observation was supplied; the actual configuration returns a null total and nonzero BLOCKED. Forecasts never become observed spend.

Final direct native audit observed an advisory HTTPS POST 200, exit 1 and four moderate affected entries from one minimatch issue, with no high/critical entries. Native SBOM generation returned 69 production components. Manual Trivy covered the final 231 unique locked Node package/version pairs, three hashed Python packages and Docker configuration; it retained one medium advisory and one low role-specific HEALTHCHECK observation. Source/explicit-public Secretlint found no secrets. Raw JSON alone cannot prove native producer execution: actual CodeQL, OCI/image scanning, Git history and hosted CI remain unverified. The current full assessment is **14 PASS / 2 N/A / 31 GAP**, without production clearance.

7. Reproducibility

From the separate project root, install locked runtime packages, generate disposable test PKI, and run:

```
python scripts/test-pki.py --flat --output artifacts
npm test
python sdk/python/test_agent_runtime.py
npm run product:check
npm run product:evaluate
npm run product:load
node scripts/product-verify.mjs --evidence artifacts/product-evaluation.signed.json --public-key
<independently-pinned-evaluator-public.pem>
node scripts/operations-check.mjs --out artifacts/new-product-operations.json
```

PKI generation requires test-only `cryptography==50.0.0`. The evaluator stores its public key but never its ephemeral private key. Pin that public key through another trusted channel; accepting the accompanying key is only a local demonstration. Implementation digests cover recursive source/configuration files and exclude private generated identities, databases, caches, archives and documentation. Digests identify bytes, not a deployed trusted host.

Run isolated tests, Linux socket acceptance and the live containment checker separately on the supported host. A nonzero blocked gate is not skipped-as-pass and must not silently fall back to TCP or an uncontained process.

8. Limits and next validation

The project has an implementation and local evidence, not production clearance. Required work includes genuine Linux containment and renewal; complete route/credential inventory with real IAM; customer and live companion/ledger acceptance; genuine off-site recovery/source fencing/privacy reconciliation; independent checkpoint/key custody; deployed monitoring/alerts/traces; native CodeQL/built-image/hosted pipeline acceptance; representative production data/load and independent security review.

The 47-row readiness assessment records each production requirement without inheriting old prototype passes. A completed engineering draft does not prove superiority over EthicalBrain or any other product, patentability, worldwide priority, regulatory acceptance or guaranteed safety.

9. Conclusion

The reference design moves company-agent authority into a durable externally checked execution path and makes its assumptions explicit. Reviewed learning changes warnings while hard permissions remain independently enforced. Reproducible local effect measurements and signed evidence make the declared behavior reviewable. Whether a company deployment is protected depends on actual credential custody, resource admission and complete mediation on its real infrastructure; those claims require the remaining deployment and independent acceptance work.